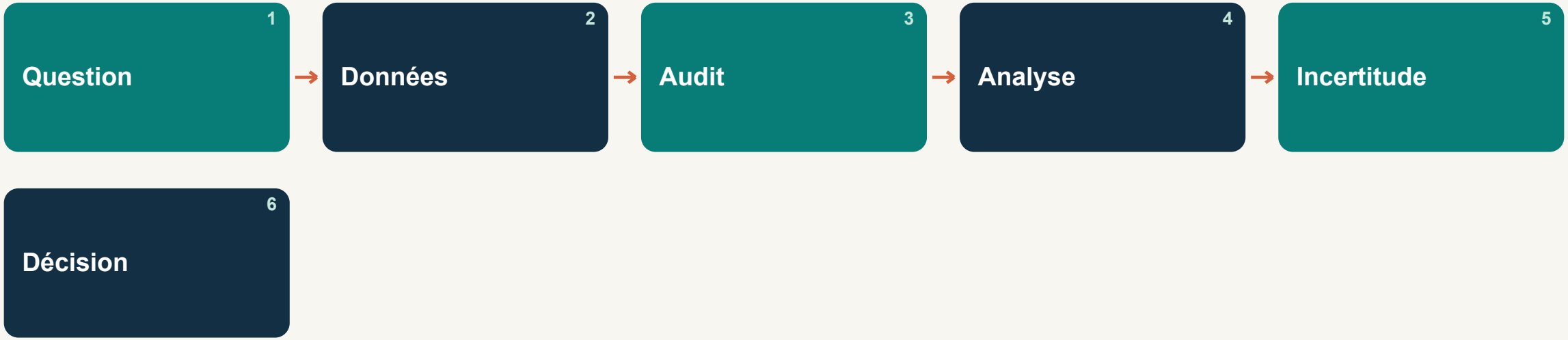
PROJET 0 → 9 · MISSION

Une question qui mérite une analyse

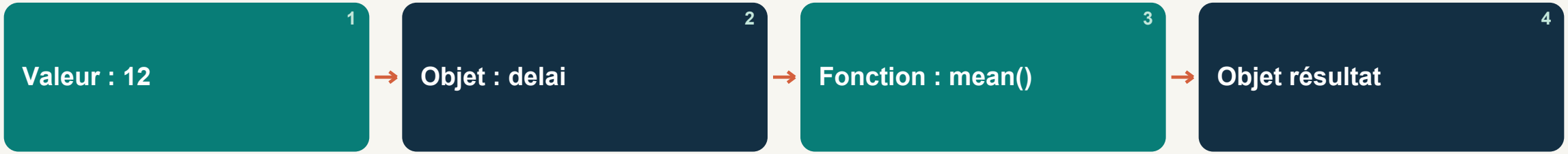


CONTRAT SCIENTIFIQUE

Ce sont des données synthétiques, pas une étude réelle ni une preuve commerciale. Le générateur contient un effet ; l'exercice porte sur l'estimation et ses limites.

01 · OBJET, FONCTION, RÉSULTAT

R est un atelier de calcul



À VOUS · 90 SECONDES

Remplacer 30 par 120. Quel résumé bouge le plus : mean() ou median() ? Expliquer en une phrase.

02 · TYPE ET STRUCTURE

Les types changent le sens

Valeur	Type / classe	Sens
12.5	numeric	Mesure quantitative
3L	integer	Entier
"A"	character	Texte
TRUE	logical	Condition vraie

PIÈGE CONCRET

Un identifiant numérique reste un identifiant : la moyenne de ticket_id ne décrit pas une performance. Satisfaction 1–5 est ordinale ; ses distances ne sont pas garanties égales.

03 · CRÉER ET INDEXER

Un vecteur, plusieurs questions

```
x <- c(12, 18, 30, 45)
x[1]                  # R commence à 1
x[c(1, 4)]
x[x > 20]             # filtre logique
sum(x > 20)           # compte TRUE
mean(x > 20)          # proportion
seq(0, 60, by = 15)
rep(c("A", "B"), each = 2)
```

TRACEZ AVANT DE LANCER

Écrire les quatre valeurs de x > 20 sur papier. Puis comparer x[2] et x[-2] : position choisie ou retirée ? Le TP01 vous demande de construire votre propre exemple.

PAUSE ACTIVE · APRÈS LE CONCEPT 04

À vous de prendre la main

PRÉDIRE → TESTER → EXPLIQUER

Écrivez une prédiction seul, exécutez un exemple différent, puis comparez avec un pair. Chacun doit pouvoir expliquer une ligne et une limite.

- 90 secondes : écrire sans lancer R.
- 3 minutes : tester et conserver une preuve.
- 2 minutes : défendre le résultat à un pair.

04 · DÉNOMINATEUR VISIBLE

NA ne veut pas dire zéro

```
x <- c(10, NA, 30)
mean(x)                    # NA
is.na(x)
sum(is.na(x))              # 1
mean(x, na.rm = TRUE)       # 20 sur 2 mesures
x[!is.na(x) & x > 15]
```

SI TOUT MANQUE

mean(c(NA_real_, NA_real_), na.rm = TRUE) donne NaN. Une fonction professionnelle définit ce cas explicitement et garde le nombre d'observations.

05 · IF, FOR ET VECTORISATION

Décider, répéter, puis simplifier

```
x <- c(12, 75, 35)
for (i in seq_along(x)) {
  print(x[i] > 40)
}
x > 40
any(x > 40)
```

ERREUR À EXPLIQUER

Pourquoi if (x > 40) est-il incorrect ici ? Écrire une version qui répond à « au moins un délai dépasse 40 » avec any().

PAUSE ACTIVE · APRÈS LE CONCEPT 06

À vous de prendre la main

PRÉDIRE → TESTER → EXPLIQUER

Écrivez une prédiction seul, exécutez un exemple différent, puis comparez avec un pair. Chacun doit pouvoir expliquer une ligne et une limite.

- 90 secondes : écrire sans lancer R.
- 3 minutes : tester et conserver une preuve.
- 2 minutes : défendre le résultat à un pair.

06 · ENTRÉE, SORTIE, CAS LIMITE

Une fonction est un contrat

```
safe_mean <- function(x) {
  if (!is.numeric(x)) stop("Type incorrect")
  if (any(!is.finite(x) & !is.na(x))) stop("Inf")
  if (all(is.na(x))) return(NA_real_)
  mean(x, na.rm = TRUE)
}
```

UN CONTRAT TESTABLE

Prédire les sorties pour c(10, NA, 30), numeric(0), c(NA, NA), Inf et "10". Puis vérifier chaque cas.

07 · DIMENSIONS ET ORDRE

Une matrice organise un vecteur

```
v <- 1:6
m <- matrix(v, nrow = 2)
m                          # colonnes d'abord
m_lignes <- matrix(v, nrow = 2, byrow = TRUE)
dim(m)                     # 2 3
dim(m) <- c(3, 2)           # mêmes 6 valeurs
m[2, 1]
```

AVANT D'EXÉCUTER

Dessiner m après dim(m) <- c(3, 2). Modifier les dimensions ne réordonne pas le vecteur sous-jacent ; le nombre de cases doit rester 6.

08 · LIGNES OU COLONNES

apply() suit une direction

```
m <- matrix(1:6, nrow = 2)
apply(m, 1, sum)        # lignes : 9, 12
apply(m, 2, sum)        # colonnes : 3, 7, 11
colMeans(m)
rowSums(m)
```

PRATIQUE SÛRE

Sélectionner explicitement les colonnes numériques avant as.matrix() et apply(). Pour une liste ou des colonnes hétérogènes, lapply() et vapply() évitent cette coercition globale.

PAUSE ACTIVE · APRÈS LE CONCEPT 09

À vous de prendre la main

PRÉDIRE → TESTER → EXPLIQUER

Écrivez une prédiction seul, exécutez un exemple différent, puis comparez avec un pair. Chacun doit pouvoir expliquer une ligne et une limite.

- 90 secondes : écrire sans lancer R.
- 3 minutes : tester et conserver une preuve.
- 2 minutes : défendre le résultat à un pair.

09 · UNE LIGNE = UN TICKET

Importer une table sans deviner

```
d <- read.csv("data/noria_dirty.csv",
              stringsAsFactors = FALSE,
              na.strings = c("", "NA"))
dim(d)
head(d)
str(d)
summary(d)
colSums(is.na(d))
```

AVANT LA MOYENNE

Nommer l'unité d'observation, la population cible, l'unité du délai et la clé ticket_id. Un fichier qui s'ouvre sans erreur peut encore contenir des doublons et des valeurs impossibles.

10 · CHAQUE PERTE A UNE RAISON

Auditer avant d’exclure

1 **241 lignes brutes** → 2 **240 tickets uniques** → 3 **236 délais valides**

UNE EXCLUSION N’EST PAS UNE RÉPARATION

Les 236 tickets valides constituent une analyse sur cas complets pour le délai. Comparer les exclusions par groupe et discuter le risque de biais ; ne pas écrire que les données sont « propres donc sans biais ».

APRÈS LE TP02 · AVANT LES GRAPHIQUES ET TESTS

Utiliser la table analysable

```
source("R/00_utils.R", encoding = "UTF-8")
source("R/02_clean.R", encoding = "UTF-8")
nettoye <- clean_noria(d)
nettoye$audit
d <- nettoye$data
stopifnot(nrow(d) == 236)
```

CONVENTION

d désigne désormais les 236 tickets nettoyés. Le brut et ses exclusions restent conservés ; ne jamais écraser le CSV reçu.

PAUSE ACTIVE · APRÈS LE CONCEPT 11

À vous de prendre la main

PRÉDIRE → TESTER → EXPLIQUER

Écrivez une prédiction seul, exécutez un exemple différent, puis comparez avec un pair. Chacun doit pouvoir expliquer une ligne et une limite.

- 90 secondes : écrire sans lancer R.
- 3 minutes : tester et conserver une preuve.
- 2 minutes : défendre le résultat à un pair.

11 · DISTRIBUTION, PAS DÉCORATION

Le graphique doit répondre

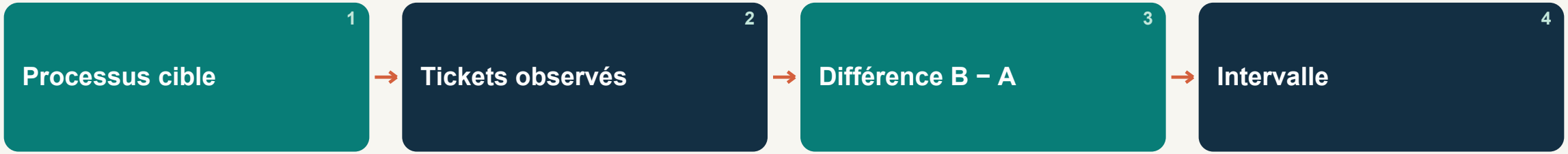
Question	Graphique	Ce que l'on inspecte
Quelle forme ?	Histogramme du délai	Asymétrie, valeurs extrêmes, échelle
A diffère de B ?	Boîtes + points par groupe	Centre, dispersion, recouvrement, n
Complexité liée au délai ?	Nuage de points	Pente, non-linéarité, groupes
Le modèle manque quelque chose ?	Résidus vs ajustés / QQ	Structure, variance, queues

TOUJOURS LÉGENDER

Unité sur chaque axe, titre qui dit ce qui est représenté, effectifs dans la légende ou la table. Éviter les axes trompeurs et une moyenne seule sans dispersion.

12 · POPULATION, ÉCHANTILLON, EFFET

Du hasard à l’incertitude



TROIS VERDICTS POSSIBLES

Évidence d’une différence ; absence d’évidence suffisante ; bénéfice opérationnel incertain. Aucun n’équivaut automatiquement à « déployer ».

13 · CONTRASTE ANNONCÉ B − A

Welch : comparer deux moyennes

```
a <- d$resolution_minutes[d$workflow == "A"]
b <- d$resolution_minutes[d$workflow == "B"]
test <- t.test(x = b, y = a,
               alternative = "two.sided",
               conf.level = 0.95)
mean(b) - mean(a)
test$conf.int
test$p.value
```

SENS DU RÉSULTAT

La formule t.test(delai ~ workflow) utilise l'ordre des niveaux ; elle peut afficher A − B. Ici x = b, y = a évite l'ambiguïté.

PAUSE ACTIVE · APRÈS LE CONCEPT 14

À vous de prendre la main

PRÉDIRE → TESTER → EXPLIQUER

Écrivez une prédiction seul, exécutez un exemple différent, puis comparez avec un pair. Chacun doit pouvoir expliquer une ligne et une limite.

- 90 secondes : écrire sans lancer R.
- 3 minutes : tester et conserver une preuve.
- 2 minutes : défendre le résultat à un pair.

14 · CLASS, ATTRIBUTES, MÉTHODES

Le résultat statistique est un objet

```
class(test)
names(test)
attributes(test)
str(test, max.level = 1)
test$estimate
unname(test$conf.int)
test$p.value
```

POUR LES MODÈLES

lm() produit un objet de classe lm. Les méthodes coef(), confint(), fitted() et residuals() décrivent sa structure sans copier à la main le texte imprimé.

15 · AJUSTER SUR LA COMPLEXITÉ

La régression ajoute une question

```
d$workflow <- factor(d$workflow, levels = c("A", "B"))
modele <- lm(resolution_minutes ~ workflow + complexity + team,
             data = d)
coef(modele)
confint(modele)
summary(modele)
attributes(modele)
```

AU-DELÀ DU COEFFICIENT

Welch principal et modèle ajusté secondaire sont fixés par le protocole. Rapporter les deux résultats, sans choisir selon leur p-valeur ; les coefficients restent conditionnés aux hypothèses.

PAUSE ACTIVE · APRÈS LE CONCEPT 16

À vous de prendre la main

PRÉDIRE → TESTER → EXPLIQUER

Écrivez une prédiction seul, exécutez un exemple différent, puis comparez avec un pair. Chacun doit pouvoir expliquer une ligne et une limite.

- 90 secondes : écrire sans lancer R.
- 3 minutes : tester et conserver une preuve.
- 2 minutes : défendre le résultat à un pair.

16 · DIAGNOSTICS ET LIMITES

Un bon modèle accepte la critique

```
par(mfrow = c(1, 2))
plot(fitted(modele), residuals(modele),
     xlab = "Délai ajusté (min)", ylab = "Résidu (min)")
abline(h = 0, lty = 2)
qqnorm(residuals(modele))
qqline(residuals(modele))
par(mfrow = c(1, 1))
```

INTERPRÉTATION HONNÊTE

Un graphique rassurant ne prouve pas une hypothèse. En cas de doute : décrire la fragilité, tester une sensibilité annoncée, ou restreindre la conclusion.

17 · BOOTSTRAP RAISONNÉ

Extension : refaire le tirage

```
set.seed(42)
delta <- replicate(1000, {
  mean(sample(b, length(b), replace = TRUE)) -
    mean(sample(a, length(a), replace = TRUE))
})
quantile(delta, c(0.025, 0.975))
```

OPTION AVANCÉE, HORS NOYAU

Comparer l'IC bootstrap à Welch, conserver la graine et expliquer pourquoi un désaccord mérite une enquête plutôt qu'un choix opportuniste.

18 · CALCUL DÉCOMPOSABLE

MapReduce commence sur papier



À PRÉSERVER

Transporter des sommes et des effectifs, pas seulement des moyennes. Les additions se combinent ; une médiane locale n’est pas suffisante pour retrouver la médiane globale.

PAUSE ACTIVE · APRÈS LE CONCEPT 19

À vous de prendre la main

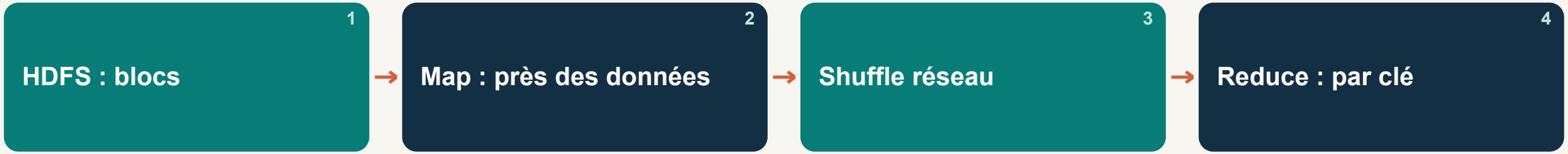
PRÉDIRE → TESTER → EXPLIQUER

Écrivez une prédiction seul, exécutez un exemple différent, puis comparez avec un pair. Chacun doit pouvoir expliquer une ligne et une limite.

- 90 secondes : écrire sans lancer R.
- 3 minutes : tester et conserver une preuve.
- 2 minutes : défendre le résultat à un pair.

19 · STOCKAGE, ORCHESTRATION, CALCUL

Hadoop n’est pas une fonction R



CE QUE LE TP PROUVE

Le TP05 vérifie l’algorithme en mémoire dans un processus R. Il ne teste ni HDFS, ni parallélisme, ni reprise après panne, ni déploiement RHadoop. Un cluster est inutile pour nos 241 lignes.

20 · PASSERELLE VERS LE PLAN OFFICIEL

Lire une API RHadoop historique

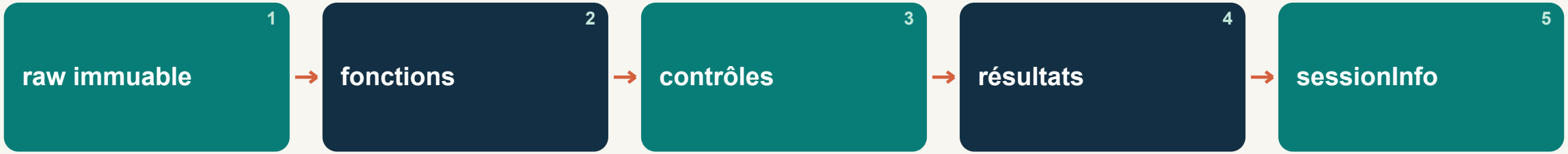
```
# PSEUDOCODE historique, pas executable ici
mapreduce(
  input = "hdfs:///tickets",
  map = function(key, rows) emettre_somme_et_n(rows),
  reduce = function(key, values) combiner_sommes_et_n(values)
)
# Finaliser : total_delai / total_effectif
```

ADAPTATION À VALIDER

La mise en œuvre RHadoop réelle demandée par le plan n'est pas réalisée ici. Le formateur doit faire approuver la substitution pratique locale ; les notions Hadoop/HDFS restent traitées.

21 · REPRODUCTIBILITÉ À PETITE ÉCHELLE

Livrer comme un collègue fiable



GIT EST UNE EXTENSION

Un commit explique une décision et permet de comparer. Ne jamais publier données personnelles, secrets, captures de comptes ou noms d’étudiants dans un dépôt public. Un ZIP local reste un livrable valable.

22 · LE PROJECTEUR EST À VOUS

Votre résultat tient en une page

Bloc	Preuve attendue
Question	Population, mesure, contraste B − A
Qualité	Effectifs avant/après et causes d’exclusion
Résultat	Estimation, IC, graphique et dénominateurs
Interprétation	Ampleur, incertitude, hypothèses

QUESTION DE SOUTENANCE

« Quelle observation nouvelle pourrait vous faire changer d’avis ? » Une bonne analyse peut proposer un prochain test au lieu de recommander immédiatement un déploiement.

PAUSE ACTIVE · APRÈS LE CONCEPT 23

À vous de prendre la main

PRÉDIRE → TESTER → EXPLIQUER

Écrivez une prédiction seul, exécutez un exemple différent, puis comparez avec un pair. Chacun doit pouvoir expliquer une ligne et une limite.

- 90 secondes : écrire sans lancer R.
- 3 minutes : tester et conserver une preuve.
- 2 minutes : défendre le résultat à un pair.